MILESTONE: Alpha
TEAM NAME: Team Gathering
GAME NAME: Forest of the Food
NAME: Minjae Kyung
JOBS: Technical Director
DEGREE PROGRAM: RTIS
GAME DESCRIPTION: The forest of food is player finds,steals and collects ingredient of foods and makes favorite seasonal food. And player should satisfy fairy of the forest to escape the forest of food.
LINKS: https://bitbucket.org/llttime/llttime/src/master/

Work Done

I've done Object collision check and resolve collision function(CollisionInteractWith) of sub classes

In Object class :

```
Direction CollisionCheck(const Object* other, double dt);
virtual void CollisionInteractWith(Object*, Direction);
```

```
void HitboxDraw(Camera CameraMatrix);
void HitboxUpdate();
```

Overall look of collision

```
Direction Object::CollisionCheck(const Object* other, double dt) //Collision direction between object(notD is not Collisioned)
{
    if (hitbox.isOverlapped(other->hitbox))
    {
        Vector2D relativeVelocity = velocity - other->velocity;
        Vector2D collisionEdge;

        relativeVelocity *= dt;

        Direction edgeDirection = Direction::notD;

        if(other->hitbox.isPointInside(hitbox.GetPositionOf(Direction::leftUp)))
        {
            collisionEdge = hitbox.GetPositionOf(Direction::leftUp);
            edgeDirection = Direction::leftUp;
        }
        else if (other->hitbox.isPointInside(hitbox.GetPositionOf(Direction::rightUp)))
        {
            collisionEdge = hitbox.GetPositionOf(Direction::rightUp);
            edgeDirection = Direction::rightUp;
        }
        else if (other->hitbox.isPointInside(hitbox.GetPositionOf(Direction::rightDown)))
        {
            collisionEdge = hitbox.GetPositionOf(Direction::rightDown);
            edgeDirection = Direction::rightDown;
        }
        else if (other->hitbox.isPointInside(hitbox.GetPositionOf(Direction::leftDown)))
        {
            collisionEdge = hitbox.GetPositionOf(Direction::leftDown);
            edgeDirection = Direction::leftDown;
        }
```

```
            if (isCrossing({ collisionEdge, collisionEdge - relativeVelocity },
                { other->hitbox.GetPositionOf(Direction::leftUp), other->hitbox.GetPositionOf(Direction::rightUp) })
                &&
                (edgeDirection == Direction::rightDown || edgeDirection == Direction::leftDown))
            {
                return Direction::up;
            }
            else if (isCrossing({ collisionEdge, collisionEdge - relativeVelocity },
                {other->hitbox.GetPositionOf(Direction::rightUp), other->hitbox.GetPositionOf(Direction::rightDown) })
                &&
                (edgeDirection == Direction::leftUp || edgeDirection == Direction::leftDown))
            {
                return Direction::right;
            }
            else if (isCrossing({ collisionEdge, collisionEdge - relativeVelocity },
                {other->hitbox.GetPositionOf(Direction::rightDown), other->hitbox.GetPositionOf(Direction::leftDown) })
                &&
                (edgeDirection == Direction::rightUp || edgeDirection == Direction::leftUp))
            {
                return Direction::down;
            }
            else if (isCrossing({ collisionEdge, collisionEdge - relativeVelocity },
                {other->hitbox.GetPositionOf(Direction::leftDown), other->hitbox.GetPositionOf(Direction::leftUp) })
                &&
                (edgeDirection == Direction::rightUp || edgeDirection == Direction::rightDown))
            {
                return Direction::left;
            }


            return Direction::center;
        }

        return Direction::notD;
    }
```

In Player Class

```
void Player::CollisionInteractWithEnviroment(Object* obj, Direction direction)
{
    constexpr double correction = 2.;
    switch (direction)
    {
    case Direction::up:
        position.y += obj->GetHitbox().GetPositionOf(Direction::up).y - GetHitbox().GetPositionOf(Direction::down).y + correction;
        velocity.y = 0;
        blocked.isDown = true;
        break;
    case Direction::down:
        position.y += obj->GetHitbox().GetPositionOf(Direction::down).y - GetHitbox().GetPositionOf(Direction::up).y - correction;
        velocity.y = 0;
        blocked.isUp = true;
        break;
    case Direction::left:
        position.x += obj->GetHitbox().GetPositionOf(Direction::left).x - GetHitbox().GetPositionOf(Direction::right).x - correction;
        velocity.x = 0;
        blocked.isRight = true;
        break;
    case Direction::right:
        position.x += obj->GetHitbox().GetPositionOf(Direction::right).x - GetHitbox().GetPositionOf(Direction::left).x + correction;
        velocity.x = 0;
        blocked.isLeft = true;
        break;
    }
}

void Player::CollisionInteractWithItem(Item* obj, Direction )
{
    inventory.AddItem(obj);
}
```

In Knife class

```
if (timer > 0)
{
    position = objManager.player.GetPosition() + objManager.player.GetHotSpot(1);
    HitboxUpdate();

    Direction temp;
   for(int i = 0; i< objManager.animalObjects.GetAnimals().size();i++)
   {
       Creature* copy = objManager.animalObjects.GetAnimals()[i];
       temp = CollisionCheck(copy, dt);
       std::cout << temp;

       if (temp != Direction::notD)
       {
           std::cout << "Direction : " << temp << std::endl;
           std::cout << "[Animal] ₩n";
           CollisionInteractWith(copy, temp);
           objManager.animalObjects.Animal_Erase(copy,objManager);


       }
   }
    timer -= dt;
}
```

I also made ObjectManager

```
#pragma once

#include "../Game/Player.h"
#include "../Game/Item.h"
#include "../Game/Creature.h"
#include "../Game/Environment.h"

class GameObjectManager {
public:
    void Load(unsigned int season);
    void Unload();

    void UpdateAll(double dt);
    void DrawAll(TransformMatrix& cameraMatrix, unsigned int season);

    GameObjectManager(Player player)noexcept :player(std::move(player), 0) {}

    Player player;
    Items itemObjects;
    Animals animalObjects;
    Trees treeObjects;
};
```